SOLVER GUIDE • MHD-WAAM THESIS • ALL PHASES

Turning a Laser Solver into an Arc Solver

Which code we are editing, why that one, what it already does, what we have changed so far, and what is still to come

Sabbir Dipto | laserbeamFoam vendored in `waam-mhd-thesis` *| OpenFOAM ESI v2506 | Al–5%Mg GMAW-WAAM*

READ THIS FIRST

We are not writing a solver. We are making **seven small, tracked changes** to an existing one — laserbeamFoam — which already contains almost every piece of physics this thesis needs, including the magnetic module. What it does not have is an **arc**: it heats with a ray-traced laser, and it has no welding current.

Only four of the seven touch solver source at all; the other three are dictionary entries and boundary conditions. **Two are done and verified.** Five remain, and one of those five — injecting the welding current — is the actual contribution of the thesis.

Everything below states where each piece of code lives, what it does, and whether it has been tested.

1. Which solver, and how to find it

Solver	**laserbeamFoam** — a volume-of-fluid melt-pool code written for laser additive manufacturing
Upstream	`github.com/laserbeamfoam/LaserbeamFoam`, branch `OpenFoam_com_main`, commit `7578023c`
Our repository	`github.com/sabbirdipto/waam-mhd-thesis` (**private**) — one repository holding everything: cases, docs, materials, runs, tools *and* the solver
Our clone	`~/thesis/src/lbf-com` — **vendored**, not a nested clone. It has no `.git` of its own; its files are tracked by the thesis repository like any others
Baseline	Upstream `7578023c`, recorded in `src/lbf-com-PROVENANCE.txt` beside the vendored tree
History archive	`github.com/sabbirdipto/LaserbeamFoam`, branch `waam-mhd` — **frozen**. Holds the 537 upstream commits plus `2a75ee8` and `a248196`. Never pushed to again
Solver source	`~/thesis/src/lbf-com/applications/solvers/laserbeamFoam/`
Magnetic module	`~/thesis/src/lbf-com/src/MTHD/`
Built binary	`~/OpenFOAM/sabbirdipto-v2506/platforms/linux64GccDPInt32Opt/bin/laserbeamFoam`
OpenFOAM	ESI **v2506**, loaded with the `of2506` shell function. NOT the Foundation OF10 install — the APIs are incompatible

Why this solver and not another

CANDIDATE	VERDICT	REASON
laserbeamFoam `OpenFoam_com_main`	**chosen**	Has VOF, enthalpy-porosity melting, Marangoni, CSF surface tension AND a working magnetohydrodynamics module with its own validation tutorial. The newest MTHD of any branch, 484 lines

CANDIDATE	VERDICT	REASON
laserbeamFoam `OpenFoam_Org_main`	rejected	The Foundation-fork branch has no MTHD at all. This is why the project moved from OpenFOAM 10 to ESI v2506
DEDFoamv2	rejected	Recommended at first, then checked against the other branches. Older MTHD (439 lines) and its one unique feature is a Lagrangian *powder* cloud. You deposit wire
ORNL AdditiveFOAM	rejected	Inspected the source: only a steady `rhoSimpleFoam` variant compiles. Zero matches for alpha, melt, Marangoni, Darcy or VOF
Write from scratch	rejected	Six months of solver development before any thesis physics. Everything below marked *free* would have had to be written and validated

2. What the solver already gives us

Ten pieces of physics ship working and validated. This is the entire reason for choosing an existing solver: each of these would otherwise be weeks of work and a separate verification problem.

PHYSICS	WHERE IT LIVES	STATUS
VOF interface tracking (isoAdvector)	`alphaEqnSubCycle.H`	free
Enthalpy-porosity melting, Voller-Prakash	`TEqn.H` lines 89-103	free
Latent heat release	`TEqn.H` line 29	free
Darcy momentum sink in the mushy zone	`UEqn.H` lines 13-14	free
Surface tension, CSF, balanced-force	`UEqn.H` line 87 + `pEqn.H`	free
Marangoni (thermocapillary) stress	`UEqn.H` line 26	free
Boussinesq buoyancy	`updateProps.H` line 16	free
Lorentz force J × B	`src/MTHD/`, added to `UEqn.H`	free
Joule heating	`TEqn.H`, as `Joulesource`	free
Conductivity and permeability blended across the interface	`src/MTHD/mthdModel.C`	free

*Verified on this machine in Phase 1: the Hartmann analytical benchmark reproduces to **1.39%**, and MHD runs alongside melting and Marangoni for 6,906 steps without collapsing.*

3. What it does not have: the gap

laserbeamFoam is a *laser* code. Gas metal arc welding differs in four ways, and every one of them has to be added.

	LASER (what the code does)	GMAW-WAAM (what we need)
Heat source	Ray-traced beam with Fresnel absorption off the free surface	A moving arc, modelled as Goldak's double ellipsoid
Electric current	**None.** A laser carries no current, so the only J is motional, sigma(u × B)	**135 A injected** through the pool. |J| ~ 1.07e7 A/m^2
Material addition	None, or a Lagrangian powder cloud	Wire fed at 0.15 m/s, melting into droplets that fall into the pool
Surface pressure	Vaporisation recoil — keyhole physics	Arc pressure and droplet impact. No keyhole

THE ONE THAT DECIDES THE THESIS

The missing current is not one gap among four — it is the whole problem. Two different couplings share the name *Lorentz force*:

STIRRING is J_{arc} × B, with current from the welding arc. It is *linear* in B and it **drives** flow. **BRAKING** is sigma(u × B) × B, with current from the melt's own motion. It is *quadratic* in B and it **damps** flow.

At 20–50 mT — the range every experiment uses — stirring is **200 to 500 times** larger. The two cross only at 9.8 T.

The shipped MTHD module has only the braking term, because a laser has no current to inject. So the solver as delivered is two to three orders of magnitude away from the dominant physics of magnetised GMAW, at every field strength anyone applies. Injecting the welding current is therefore not a refinement — without it the model cannot show the effect at all.

4. How to read this solver

Before changing anything it is worth knowing the shape of one timestep. From `laserbeamFoam.C`, with the line numbers as they were before our edits:

```
112: while (runTime.run())                  // time loop
142:     while (pimple.loop())              // outer correctors
149:         alphaEqnSubCycle.H             VOF: move the interface
158:         updateProps.H                  properties from T
                 |-> updateKappaCp.H        kappa, cp    <-- EDIT 1 HERE
173:         UEqn.H                         momentum  (HAS fvOptions)
178:         TEqn.H                         energy    (NO fvOptions)  <-- EDIT 2
181:         while (pimple.correct())
183:             pEqn.H                     pressure
```

The two edits made so far sit at lines 158 and 178. Everything still to come attaches to one of these same points.

The pattern the solver uses for optional physics

Every optional piece of physics enters as a named source on the right-hand side of an equation, collapsing to zero when switched off. The energy equation shows it clearly:

```
fvScalarMatrix TEqn
(
    fvm::ddt(rhoCp,T) + fvm::div(rhophicp, T)
  - fvm::laplacian(kappa, T)
  ...
  ==
    laser.deposition()      // zero when the laser power table is zero
  + arcSource               // OUR EDIT -- zero without an arc dictionary
  + Joulesource             // zero when electromagnetics is off
  + (Qrad_expl - Qv)*gradAlphaDamper
  - TRHS
);
```

This is the pattern every change follows. Add a term, make it zero by default, and every existing case keeps running unchanged. That is what lets a regression test prove a patch is safe.

5. Six things we learned by reading the source

Each of these changed the plan. None could have been guessed; all came from reading the code rather than assuming.

1. TEqn has NO fvOptions. UEqn does.

The plan had assumed the arc heat source could be a `codedFvOption` — a snippet in a case dictionary, no solver edit. But `TEqn.H` never calls `fvOptions`, so a coded source would be **silently ignored**: the case runs, the arc does nothing, and nothing reports why.

`UEqn.H` line 71 *does* call it. So momentum sources (plasma drag) can stay dictionary snippets; energy sources cannot.

2. kappa is computed BEFORE T is updated, always.

`updateProps.H` runs at line 158, `TEqn.H` at 178. So the conductivity written to disk was evaluated from an *earlier* temperature than the T written beside it.

This is pre-existing behaviour, not something we introduced — with a single conductivity polynomial it is simply invisible, because you cannot work backwards to a temperature. It matters because any test assuming fields in a time directory are simultaneous will fail for the wrong reason.

3. Surface forces must appear in BOTH UEqn and pEqn.

The vaporisation recoil pressure `pVap` appears at `UEqn.H` line 88 **and** `pEqn.H` line 32. That duplication is the *balanced-force* requirement: an interface force and the pressure gradient must be discretised on the same faces, or you get spurious currents at the free surface — which is exactly where the Marangoni result lives.

Arc pressure will follow this pattern. A `codedFvOption` would add a cell-centred force instead and break the balance.

4. epsilon1 is the liquid fraction, and it IS Zhao's f_l.

Confirmed from three places: latent heat is released proportional to d(epsilon1)/dt (`TEqn.H` line 29); the Darcy sink is $A(1-\text{epsilon1})^2/(\text{epsilon1}^3+\text{small})$ (`UEqn.H` 13-14), which is Zhao Eq. 7 verbatim; and Marangoni acts only where epsilon1 > 0.

So the solver's melting model and the paper's are the same model.

5. Elec_conductivity is a SINGLE SCALAR per phase.

It cannot be made temperature dependent through the dictionary. Our $\sigma_e(T)$ runs 1.64e7 at 300 K down to 3.65e6 at the liquidus — a factor of 4.5 — and all of it collapses to one number.

We use the liquidus value, because the melt carries the MHD. But the solid is then understated 4.5-fold, which will matter in Phase 6 when injected current has to flow through the substrate. Expect a code change there too.

6. poly_kappa is ONE polynomial per phase.

Aluminium's thermal conductivity **halves** on melting, 162 to 81 W/m/K across 91 K. No smooth degree-7 polynomial can represent a step: the best full-range fit is **84% wrong at the liquidus**.

That error sits at the solidification front, where Darcy damping kills velocity so conduction dominates locally, and where ExaCA reads the cooling rate that the whole microstructure prediction depends on. This became Edit 1.

The two replacement polynomials were not fitted to conductivity data, which for liquid Al-Mg barely exists. They come from the **Wiedemann-Franz law**, k = L0 sigma T with L0 = 2.44e-8 W ohm/K^2, applied to the electrical conductivity built in Phase 2 and cross-validated to 1%. Both live in `~/thesis/materials/transportProperties` as `poly_kappa_solid` and `poly_kappa_liquid`.

6. What has been changed so far

#	CHANGE	FILES	STATUS
1	Two-branch thermal conductivity	`createFields.H`, `updateKappaCp.H`	**DONE**
2	Goldak arc heat source	`createFields.H`, `updateArcSource.H` (new), `TEqn.H`, `laserbeamFoam.C`	**DONE**

Four files modified, one created: 207 insertions and 8 deletions, plus a 73-line new file. Originally two commits, `2a75ee8` and `a248196`, now vendored into the thesis repository.

Edit 1 — two-branch thermal conductivity

The problem. One polynomial cannot represent aluminium's conductivity step at melting. **The fix.** Two optional dictionary entries, blended by liquid fraction, following the pattern the solver already uses for emissivity at `TEqn.H` line 15.

```
// ~/thesis/src/lbf-com/applications/solvers/laserbeamFoam/updateKappaCp.H

const scalar aM = max(min(alpha1I[celli], 1.0), 0.0);   // metal fraction

scalar kappaMetal;
if (twoBranchKappa)
{
    const scalar eL =                                   // liquid fraction
        max(min((TI[celli] - TsolidusMetal)
               /(TliquidusMetal - TsolidusMetal), 1.0), 0.0);

    kappaMetal = eL*polykappa_mL.value(TI[celli])
               + (1.0 - eL)*polykappa_mS.value(TI[celli]);
}
else
{
    kappaMetal = polykappa_m.value(TI[celli]);   // EXACT original expression
}

kappaI[celli] = aM*kappaMetal + (1.0 - aM)*polykappa_g.value(TI[celli]);
```

Two nested blends: solid against liquid by melt state, then metal against gas by VOF fraction.

WHY THE else BRANCH EXISTS, AND WHAT IT COST TO LEARN

The first version had no branch — it computed `eL*k + (1-eL)*k` with both branches the same polynomial. Algebraically that is exactly `k`. In floating point it differs by about one unit in the last place, roughly 5% of the time.

The regression case amplified that **single last-bit difference into a 10% velocity change**, because melt-pool flow is chaotic. Writing the original expression verbatim in the `else` branch makes the legacy path bit-identical, which is what lets a regression test distinguish *unchanged* from *barely changed*.

VERIFICATION	RESULT
Worst error, anywhere in 300–2000 K	**84% → 2.6%**
2469 fully solid cells	exact to 3.7e-10
675 fully liquid cells	exact to 5.4e-10
Gate A, Hartmann, non-chaotic	1.3869e-02 — bit-identical
Gate B, Plate2D, chaotic	all five pool statistics bit-identical

Edit 2 — Goldak arc heat source

Zhao Eq. 27. Two half-ellipsoids sharing a centre that moves with the torch, narrow in front and wide behind:

```
q = 6 sqrt(3) f Q / (a b c pi sqrt(pi))
    * exp( -3 [ (x-x0-Vt)^2/a^2 + (y-y0)^2/b^2 + (z-z0)^2/c^2 ] )
```

with a = a_f, f = f_f ahead of the torch and a = a_r, f = f_r behind it. Full detail is in the companion note *The Arc Heat Source*; the essentials are the three corrections the bare formula needs.

CORRECTION	WHY	COST IF OMITTED
Skip cells above the surface	Goldak is a HALF ellipsoid in depth. The normalisation is correct only below the surface	**Exactly 2x** the arc power
Gate on alpha.metal	Part of the ellipsoid sits in argon, which has rho = 1.6 and c_p = 520	Absurd gas temperatures that look like a physical result
Halve the target on a half domain	A half domain contains half the arc	**2x** again

Verified by numerical integration before the code was written: 1512.0 W over a full domain, 756.0 W over a half, both exact. The solver reports its own delivered power at run time.

7. Every file touched, and what changed in it

The two edits above land in **five files**: four that already existed and one that is new. Nothing else in the clone is modified — no build scripts, no library code, no tutorials. This section is the complete inventory, so the diff can be checked against it.

FILE	STATUS	WHAT CHANGED	LINES
`createFields.H`	modified	Two blocks appended, one per edit. Declares every new symbol.	**+100**
`updateKappaCp.H`	modified	The metal/gas blend replaced with the two-branch version	9 → 33
`updateArcSource.H`	**NEW FILE**	The entire Goldak source, recomputed each timestep	**73**
`TEqn.H`	modified	One term added to the right-hand side	+1
`laserbeamFoam.C`	modified	One include, placed before TEqn.H	+1

*126 lines added to existing files, 73 in the new one: **199 net**. Everything but one block is additive, which is why the legacy path can still be bit-identical. Checked against the working tree, not inferred from the patch scripts.*

> **THE NET FIGURES ABOVE ARE NOT WHAT GIT WILL PRINT**
>
> Run as a diff, the same change reports `5 files changed, 207 insertions(+), 8 deletions(-)`. Both describe this change correctly; they count differently.
>
> A diff counts a *modified* line as one deletion plus one insertion, never as zero, so gross totals always exceed the net. And the nine-line block in `updateKappaCp.H` becomes +32/-8, not +33/-9, because its first line `kappaI[celli] =` is identical before and after: diff keeps it as context rather than counting it twice.
>
> So the arithmetic that reconciles them is 207 - 8 = 199.

7.1 createFields.H — two appended blocks

Both blocks are inserted *after* a named anchor line rather than at a line number, so the patch survives upstream changes elsewhere in the file. The anchors are:

```
// Block A anchor -- the LAST line of the existing property reads:
const Polynomial<8> polykappa_g(transportPropertiesGas.lookup("poly_kappa"));

// Block B anchor -- the LAST line of Block A:
const Polynomial<8> polykappa_mL(readMetalKappa("poly_kappa_liquid"));
```

BLOCK	DECLARES	LINES
A — kappa `Edit 1`	`readMetalKappa` lambda, `polykappa_mS`, `polykappa_mL`, `TsolidusMetal`, `TliquidusMetal`, `twoBranchKappa`, and a startup line reporting which mode was selected	30
B — arc `Edit 2`	`arcDict` (reads `constant/caseParameters` with `READ_IF_PRESENT`), `arcOn`, twelve arc scalars, the `arcSource` field itself, and a startup block echoing the geometry and target power	70

THE TWO PATCHES HAVE AN ORDER, AND SO DOES UNDOING THEM

Block B anchors on the last line of Block A. The arc patch therefore **cannot be applied first** — its anchor does not exist yet, and the patcher refuses rather than guessing.

The reverse matters more. Each patcher saved its own backup before writing: `createFields.H.orig` is the file *before* either edit, `createFields.H.gbak` is the file *after* kappa but before the arc. So undo the arc first, then kappa.

Reverting kappa first restores `.orig` and deletes it; the later arc revert then writes back `.gbak` — which still contains Block A. The tree ends up kappa-patched while you believe it is clean. Git is the real safety net here; the backups are for quick local iteration only.

7.2 updateKappaCp.H — the only replacement

This is the one place where existing lines were removed. The original blended metal and gas by `alpha1` in a single expression:

```
kappaI[celli] =
    (   max(min(alpha1I[celli], 1.0), 0.0)
       *(polykappa_m.value(TI[celli]))  )
  + (   (1.0 - max(min(alpha1I[celli], 1.0), 0.0))
       *polykappa_g.value(TI[celli])    );
```

BEFORE -- 9 lines. One polynomial for the metal, whatever its state.

The replacement keeps that metal/gas blend untouched and only changes what the *metal* term evaluates to, choosing between two polynomials by liquid fraction. The full text is in section 6; the shape of the change is:

```
const scalar aM = max(min(alpha1I[celli], 1.0), 0.0);   // unchanged
scalar kappaMetal;
if (twoBranchKappa) { ...blend mS and mL by eL(T)... }
else                { kappaMetal = polykappa_m.value(TI[celli]); }
kappaI[celli] = (aM*kappaMetal) + ((1.0 - aM)*polykappa_g.value(TI[celli]));
```

AFTER -- 33 lines. The else branch holds the original expression verbatim, which is what makes the legacy path bit-identical rather than merely equal.

7.3 updateArcSource.H — the new file

73 lines at `~/thesis/src/lbf-com/applications/solvers/laserbeamFoam/updateArcSource.H`. It is a header included into the timestep loop, not a compilation unit, so it needs **no entry in Make/files** and the build system was not touched. It is wrapped in its own brace block so its locals do not leak into the loop, and the whole body sits behind `if (arcOn)` — no caseParameters, no arc, zero field, bit-identical run.

7.4 TEqn.H and laserbeamFoam.C — one line each

```
// TEqn.H -- arcSource joins the RHS as a peer of the laser
      laser.deposition()
+   + arcSource
    + Joulesource

// laserbeamFoam.C -- the field must be filled BEFORE it is read
+   #include "updateArcSource.H"
    #include "TEqn.H"
```

The include goes immediately before `TEqn.H`*, not after: the torch has moved since the last timestep, so* `arcSource` *has to be recomputed at the current time before the energy equation reads it. Placed after, the pool would be heated by the previous position — a lag of one timestep that no compiler or solver would ever complain about.*

7.5 Files changed outside the solver

These are not solver source, but they are part of the same work and the solver will not do anything correct without them.

FILE	STATUS	WHAT CHANGED
`~/thesis/materials/make_transport.py`	**NEW**, then modified	Builds the property set from the Phase 2 fits. Later edited so the gas `Elec_conductivity` emits a `@sigma_e_gas@` token instead of a fixed number.
`~/thesis/materials/transportProperties`	**NEW** (generated)	The single source of truth for material data, including `poly_kappa_solid` and `poly_kappa_liquid` that Edit 1 reads. Copied into every case, never symlinked.
`~/thesis/cases/generate_cases.py`	modified, 7 edits	Installs the property set per case and fingerprints it as `properties_sha`; refuses to compare cases built on different property sets. Derives `sigma_e_gas` from the metal value, and states the arc origin `arc_x0/y0/z0` that Edit 2 reads.
`~/thesis/cases/caseMatrix.csv`	modified twice	Conductivity sweep re-centred on the measured 1e-6 floor and two cases renamed; then the Phase 2 revision — twelve magnetic cases disabled, a negative control added, two noise twins added, notes rewritten against measured thresholds.

7.6 How to verify this section

Every count in this section was read off the working tree itself — the patched files diffed against the backups the patchers left beside them — not inferred from the scripts that wrote them. The repository is still the final authority, and it will confirm all of it from the working tree alone:

```
cd ~/thesis/src/lbf-com/applications/solvers/laserbeamFoam

diff -u createFields.H.orig  createFields.H     # both blocks, +100
diff -u updateKappaCp.H.orig updateKappaCp.H    # Edit 1,  +32 -8
diff -u TEqn.H.gbak          TEqn.H             # + arcSource
diff -u laserbeamFoam.C.gbak laserbeamFoam.C    # the include
cat     updateArcSource.H                       # new file, 73 lines
```

No git required. Totals: 207 insertions, 8 deletions across four files, plus 73 lines in one new one. If a diff disagrees with the tables above, believe the diff and not this document.

WHY THIS IS NOT A git diff, AND WHY THE BACKUPS ARE TRACKED ON PURPOSE

The solver used to be a nested clone with its own history, and this check used to read `git diff OpenFoam_com_main..waam-mhd`. It has since been **vendored** into the thesis repository: the nested `.git` was removed so one repository could hold the whole project, laid out as it is on disk.

That means there is no upstream branch left locally to diff against. The five `.orig` and `.gbak` files are what replaces it — they are the byte-exact pre-edit state of every file that was modified, so the commands above reproduce the full change set from the working tree alone. They are committed deliberately. **Do not delete them as clutter**; without them nothing in the repository records what was changed.

The 537 upstream commits are not lost, merely elsewhere: they remain on the frozen fork named in section 1, and upstream itself is public. `src/lbf-com-PROVENANCE.txt` records the baseline commit and both original commit hashes so the two can always be lined up.

8. What is still to be changed

#	CHANGE	ROUTE	PHASE	STATUS
3	**Arc pressure** on the free surface (Zhao Eqs 25-26)	Solver patch — a `pArc` field beside `pVap`, in BOTH `UEqn.H` and `pEqn.H`	3	**TO DO**
4	**Moving wire inlet** — liquid metal at 1506 K, 0.15 m/s	`codedFixedValue` boundary condition. **No solver work**	4	**TO DO**
5	**Shielding gas inlet** (Zhao Eq. 28)	`codedFixedValue`. **No solver work**	4	**TO DO**
6	**Plasma drag** on the droplet (Eq. 20)	`codedFvOption` — UEqn does support them. **No solver work**	4	**TO DO**
7	**WELDING CURRENT INJECTION**	Route A: an azimuthal H boundary profile from Ampere's law — boundary conditions only. Route B if that fails: a phi sub-solve inside MTHD	6	**TO DO**

EDIT 7 IS THE THESIS

Everything else makes the model reproduce a published result. This one makes it capable of the question the thesis asks.

Route A imposes the arc current as a magnetic field condition on the boundary, using Ampere's law: $H(r) = I_{enc}(r)/2\ pi\ r$ with $I_{enc}(r) = I[1 - exp(-r^2/2\ sigma_j^2)]$. The induction equation already in MTHD then propagates it inward. If it works it needs **no code at all** — which is why it is tried first.

Route B adds an electric potential sub-solve, div(sigma grad phi) = 0, and superposes conduction and motional currents. This edits MTHD itself.

The gate either way: set the external field to zero and compare the computed J × B against Zhao's analytic force fields, Eqs 22-24. A quantitative check against published numbers before any external field is applied.

The ledger: what needs code and what does not

	COUNT	WHAT
Ships working	**10**	VOF, melting, latent heat, Darcy, surface tension, Marangoni, buoyancy, Lorentz, Joule, interface blending
Dictionary only	**3**	Wire inlet, gas inlet, plasma drag — boundary conditions and one coded momentum option
Solver patches	**4**	Two-branch kappa (done), Goldak (done), arc pressure, current injection

The original plan claimed six items were dictionary-only. Reading the source moved three of them into code, because `TEqn` *takes no* `fvOptions` *and surface forces need the balanced-force treatment.*

9. How every change is verified

Each patch follows the same discipline, and it exists because melt-pool flow is chaotic: a round-off-level difference grows into a visible one within milliseconds, so "the answer changed a bit" carries no information on its own.

- **The patcher checks before it writes.** Every anchor must match exactly once across all files, or nothing is modified. It also verifies that new symbols are declared before the point they are used.
- **The fallback is bit-identical, not just equivalent.** Without the new dictionary entries the solver takes a branch containing the original expression verbatim.
- **Two regression cases, doing different jobs.** Hartmann is non-chaotic and *decisive* — it can attribute a difference. Plate2D is chaotic and *sensitive* — it amplifies anything real. Neither alone suffices.
- **Arithmetic is checked outside the solver first.** Goldak was integrated numerically before a line was written, which is how the factor-of-two half-ellipsoid convention was caught.
- **The solver reports what it did.** It prints its kappa mode at startup and the arc power it actually delivered, so a misconfiguration announces itself instead of being inferred from a wrong pool.

CURRENT STATUS, STATED PLAINLY

Two of seven changes are complete, verified and committed. The solver compiles, both regression gates pass bit-identically, and the arc source integrates to the correct power on paper.

The arc has never fired. No Model P case can start yet: its template holds a mesh and a control dictionary and nothing else — no initial conditions, no gravity, no schemes. Building that template is the immediate next task, and it is the only thing standing between the code as it is and the first weld pool.

So: the arithmetic is right and nothing is broken. That is not the same as producing a correct weld pool, and only the first has been established.